

DEVELOPMENT_WORKFLOW.md

Процесс разработки через **Claude Code** с человеческим контролем. Цель: Claude пишет большую часть кода, но каждая правка предсказуема, проверяема и обратима.

1. Где хранятся правила проекта

1.1 Иерархия контекста

Файл	Что содержит	Кто читает
CLAUDE.md (корень)	Главные правила: стек, границы модулей, запреты, команды, как коммитить. ≤ 200 строк.	Claude автоматически при старте
src/features/CLAUDE.md , src/tasks/CLAUDE.md , supabase/CLAUDE.md	Локальные правила подкаталога (структура feature, как писать шаблон, как писать миграцию).	Claude при работе в каталоге
docs/*.md	Архитектура, схема, безопасность — подробности.	Claude по ссылке из CLAUDE.md, люди
docs/adr/	Решения с обоснованием. Новый ADR — перед крупным изменением.	Все
.claude/commands/*.md	Слэш-команды — стандартизированные многошаговые задачи.	Claude по вызову
.claude/agents/*.md	Суб-агенты с узкой ролью (reviewer, security-auditor, test-writer).	Claude по вызову/ делегированию
.claude/settings.json	Разрешённые инструменты и хуки.	Claude Code runtime

1.2 Скелет CLAUDE.md

```
# EGE Math Platform — правила для Claude Code

## Стек
Next.js 15 App Router, React 19, TS strict, Tailwind 4, shadcn/ui, Supabase (Post
Drizzle, Zod, Vitest, Playwright, pnpm. Node 22.

## Главные принципы
1. Правильные ответы никогда не попадают на клиент. `src/tasks/templates/**` и `s
2. Любая новая таблица — с RLS и pgTAP-тестом. Схема меняется ТОЛЬКО через supaba
3. Server Action = 'use server' + Zod-валидация + guard `require*` первой строкой
4. Бизнес-логика — в features/*/service.ts (чистые функции), не в компонентах и н
5. Границы модулей — см. docs/REPOSITORY_STRUCTURE.md §4. Нарушение = ошибка сбор
6. Не добавлять зависимости без явного разрешения. Не менять config-файлы без объ
7. Не трогать content/ и supabase/seed.sql, если задача не про контент.

## Команды
pnpm dev · pnpm db:start · pnpm db:reset · pnpm check · pnpm test -- <path> · pnp

## Рабочий цикл
Прочитать задачу → предложить план (файлы, которые изменятся) → дождаться OK → re
pnpm check → написать/обновить тесты → conventional commit → PR.
Не делать больше, чем просили. Если задача неясна — задать вопрос до правок.

## Стил
Именование — REPOSITORY_STRUCTURE.md §3. Комментарии — только «почему», не «что».

## Ссылки
docs/TECH_ARCHITECTURE.md, docs/DATABASE_SCHEMA.md, docs/SECURITY.md
```

1.3 .claude/settings.json (фрагмент)

```
{
  "permissions": {
    "allow": ["Bash(pnpm *)", "Bash(git status)", "Bash(git diff *)", "Bash(git l
      "Bash(git add *)", "Bash(git commit *)", "Bash(supabase *)"],
    "deny": ["Bash(git push --force*)", "Bash(rm -rf *)", "Bash(supabase db push
      "Read(.env*)", "Read(**/*.pem)"]
  },
  "hooks": {
    "PostToolUse": [
      { "matcher": "Edit|Write",
        "hooks": [{ "type": "command", "command": "pnpm lint-staged --quiet || tr
      ],
    "Stop": [
      { "hooks": [{ "type": "command", "command": "pnpm typecheck" }] }
    ]
  }
}
```

```
]
}
}
```

Хуки гарантируют: после любой правки — форматирование и lint; при завершении задачи — `typecheck`. `db push` и `force-push` Claude выполнить не может.

2. Как Claude должен изменять код

2.1 Протокол задачи

1. **Контекст.** Claude читает `CLAUDE.md`, локальный `CLAUDE.md` каталога, релевантный `docs/`.
2. **План.** Перед правками Claude выводит: цель, список файлов (`create/modify`), затрагиваемые таблицы/RLS, тесты, риски. Разработчик отвечает «ок» или корректирует. *Для задач > 3 файлов план обязателен.*
3. **Реализация малыми шагами.** Один логический шаг — одна серия правок — один запуск `pnpm check`. Не «переписать всё, потом починить».
4. **Тесты вместе с кодом**, не после.
5. **Самопроверка.** Перед завершением Claude прогоняет чек-лист из `SECURITY.md §10`.
6. **Отчёт.** Что сделано, что не сделано, что требует решения человека.

2.2 Правила правок

- Изменять только файлы, относящиеся к задаче. Попутные «улучшения» — отдельным предложением, не в том же коммите.
- Не удалять тесты, чтобы «зазеленить». Падающий тест — либо баг в коде, либо явное обсуждение изменения контракта.
- Новая таблица/колонка → миграция + Drizzle-схема + типы (`pnpm db:types`) + RLS + pgTAP-тест — в одном PR.
- Новый шаблон задания → файл в `src/tasks/templates/`, регистрация в `registry.ts`, запись в `content/prototypes/...json`, unit-тест: 1000 генераций удовлетворяют constraints, ответ «красивый», snapshot 3 заданий.
- Новый Server Action → `schemas.ts` + `actions.ts` + `service.ts` + тест `service` без

БД + е2е при необходимости.

- Секреты, `.env`, ключи — Claude не читает и не пишет.

2.3 Слэш-команды (`.claude/commands/`)

Команда	Делает
<code>/plan <задача></code>	Только план, без правок
<code>/new-feature <name></code>	Создаёт скелет <code>features/<name>/</code> по шаблону с пустыми <code>actions/queries/service/repository/schemas/index/tests</code>
<code>/new-template <prototype_code></code>	Скелет шаблона задания + тест + JSON-запись в <code>content/</code>
<code>/migrate <описание></code>	<code>supabase migration new</code> , шаблон с RLS-блоком и pgTAP-тестом
<code>/review</code>	Запускает суб-агента <code>reviewer</code> по текущему диффу с чек-листами
<code>/security-audit</code>	Суб-агент <code>security-auditor</code> : ищет утечки ответов, отсутствие <code>guard</code> 'ов, RLS
<code>/fix-ci</code>	Читает лог падения CI (вставленный пользователем) и чинит минимально

3. Тестирование

3.1 Пирамида

Уровень	Инструмент	Что покрывает	Порог
Unit	Vitest	<code>tasks/core</code> , <code>geometry/*</code> , <code>features/*/service.ts</code> , <code>lib/*</code>	<code>tasks/core</code> , <code>geometry</code> — 90 %+; сервисы — ключевые ветки
DB / RLS	pgTAP (<code>supabase test db</code>)	Политики: <code>student</code> не читает <code>answers</code> , чужие <code>attempts</code> ; <code>teacher</code> видит только своих	Каждая таблица с RLS — минимум 1 позитивный + 1 негативный тест
	Vitest +		Критичные:

Integration	локальный Supabase	Server Actions end-to-end с реальной БД (сид)	submitAnswer, joinGroup, createAssignment, createWorksheet
E2E	Playwright	Логин, тренировка, назначение, PDF-скачивание, IDOR-сценарии	Smoke на каждый PR; полный — nightly
Snapshot	Vitest	SVG чертежей, PDF-структура (текстовый слой)	Обновление snapshot — осознанно, с объяснением в PR
Bundle	скрипт	В <code>.next/static</code> нет маркеров <code>TEMPLATE_REGISTRY</code> , <code>SUPABASE_SERVICE_ROLE</code>	Блокирующий в CI

3.2 Что Claude пишет обязательно

- Для чистых функций — table-driven тесты с граничными случаями.
- Для шаблонов заданий — property-тест (fast-check): для любых seed constraints выполняются, ответ конечен и «красив».
- Для RLS — pgTAP-негативные тесты (`throws_ok` / `results_eq` пустого набора).
- Для Server Actions — тест на невалидный вход (Zod) и на отсутствие прав (403).

3.3 Локальный прогон

```
pnpm db:start && pnpm db:reset      # локальный Supabase с миграциями и сидом
pnpm check                          # lint + typecheck + unit + pgTAP
pnpm test:e2e                       # Playwright против next dev
```

4. Commits

- **Conventional Commits**, проверяется commitlint: `feat(practice): submitAnswer with idempotency` · `fix(geometry): hidden edges for pyramid` · `db(rls): attempts policies` · `content(stereo): cylinder-volume.v1` · `docs(adr): 0003 pdf renderer` · `chore(ci): pgTAP step`.
- Scope = имя feature/модуля. Типы: `feat`, `fix`, `refactor`, `test`, `docs`, `db`, `content`,

chore, perf, sec .

- Один коммит — одно логическое изменение; миграция и код, который её использует, — в одном коммите.
- Тело коммита: «зачем», ссылка на issue. Claude добавляет трейлер Co-Authored-By: Claude <noreply@anthropic.com> .
- Pre-commit (husky + lint-staged): eslint –fix, prettier, gitleaks, typecheck затронутых файлов.
- Claude коммитит только после «зелёного» npm check . Никогда не --no-verify .

5. Ветки и откат

5.1 Git flow

main — production, защищена: только PR, обязательный CI, 1 approve
develop — staging (опционально на старте; пока команда = 1 человек, можно PR → feat/*, fix/*, db/*, content/* — рабочие ветки от main, живут ≤ 3 дней

Claude работает **только в рабочих ветках**; создание ветки — первая команда любой задачи с правками.

5.2 Откат

Ситуация	Действие
Незакоммиченная правка Claude плохая	git checkout -- <file> / git restore . ; Claude умеет git diff показать до отката
Плохой коммит в рабочей ветке	git revert <sha> (предпочтительно) или git reset --hard HEAD~1 до пуша
Плохой PR в main	git revert -m 1 <merge_sha> → PR → авто-деплой. Vercel: Instant Rollback на предыдущий деплой (секунды)
Плохая миграция БД	Forward-fix новой миграцией (expand/contract). Данные: Supabase PITR к точке до миграции (Pro). Миграции не откатываются down -скриптами — только вперёд

Плохой контент
(шаблон)

`status = archived` для шаблона; `generated_tasks` остаются с версией

Правило: **деплой кода и миграции обратно совместимы**, поэтому откат кода никогда не требует отката БД.

6. Локальный запуск

Требования: Node 22, pnpm 9, Docker Desktop (для Supabase), Supabase CLI.

```
git clone ... && cd ege-math
cp .env.example .env.local          # заполнить NEXT_PUBLIC_SUPABASE_URL/ANON_KEY
pnpm install
pnpm db:start                       # поднимает Postgres, Auth, Storage, Studio (
pnpm db:reset                       # миграции + seed (тест-аккаунты: student@tes
pnpm content:import                 # JSON-банк → БД
pnpm dev                            # http://localhost:3000
```

- Studio: `http://localhost:54323` — смотреть данные и проверять RLS через «Impersonate user».
 - Письма (magic link, reset) — Inbucket `http://localhost:54324`.
 - Sentry/PostHog в dev отключены через `env.ts` (`APP_ENV=local`).
 - Claude Code запускается в корне репо: `claude` → проверить, что `CLAUDE.md` прочитан (`/memory`).
-

7. Code review

7.1 Двухступенчатый

1. **Автоматический (Claude, суб-агент `reviewer`)** — запускается `/review` локально и GitHub Action `claude-code-action` на каждый PR. Проверяет: соответствие `CLAUDE.md`, границы модулей, наличие тестов, чек-лист безопасности, читаемость. Пишет комментарии в PR.
2. **Человеческий** — владелец продукта/тимлид. Смотрит: правильность бизнес-логики, UX, миграции, всё, что помечено `needs-human`.

7.2 Чек-лист PR (`PULL_REQUEST_TEMPLATE.md`)

```
## Что и зачем
## Как проверить
## Чек-лист
- [ ] pnpm check зелёный, e2e smoke зелёный
- [ ] Новые таблицы: миграция + RLS + pgTAP
- [ ] Server Actions: guard + Zod + Result
- [ ] Нет секретов/answers на клиенте (pnpm check:bundle)
- [ ] Тесты добавлены/обновлены; snapshot-изменения объяснены
- [ ] Обновлены docs/ADR, если менялась архитектура
- [ ] Миграция обратно совместима
## Скриншоты (UI)
## Риски / что не сделано
```

7.3 Суб-агент `reviewer` (`.claude/agents/reviewer.md` , суть)

- Роль: строгий senior-ревьюер, читает только дифф + `CLAUDE.md` + затронутые docs.
- Не правит код, только комментирует: `BLOCKER` / `SHOULD` / `NIT` .
- Обязательно проверяет 6 пунктов `SECURITY.md` §10.
- Отдельно отмечает «код, который Claude сгенерировал, но не покрыл тестом».

7.4 Правила слияния

- Squash-merge в `main` , заголовок = `conventional commit`.
- $PR \leq 400$ строк диффа (кроме контента и сгенерированных типов); больше — делить.
- Блокеры Claude-ревью должны быть закрыты или явно отклонены человеком с комментарием.

8. Типовой день разработки (пример)

1. `claude` # запуск в корне
2. `> /plan` Добавить режим `control` в `assignments` с таймером и запретом разбора до
3. [Claude выводит план: миграция `assignment_mode`, изменения в `features/assignment` RLS без изменений, тесты: `service` + `pgTAP` + `e2e control-mode`]
4. `> ок`, только таймер сделай серверным (`deadline - now`), не клиентским
5. [Claude: `git checkout -b feat/assignment-control-mode` → правки шагами → `pnpm`

6. > /review
 7. [reviewer: 1 SHOULD — нет теста на позднюю попытку; Claude добавляет]
 8. > закоммить и открой PR
 9. [feat(assignments): control mode with server-side timer ... → gh pr create]
 10. CI + Claude-action → человек ревьюит → squash-merge → Vercel deploy → Sentry
-

9. Определение готовности (Definition of Done)

Задача закрыта, когда:

- код в `main`, задеплоен на production, `/api/health` зелёный;
- тесты есть и проходят; покрытие модулей ядра не упало;
- документация/ADR обновлены при изменении архитектуры;
- в Sentry нет новых ошибок в течение 24 ч после деплоя;
- если менялась схема — резервная копия до миграции сделана (автоматически GitHub Action `backup.yml` перед `deploy-db.yml`).